PONTIFÍCIA UNIVERSIDADE CATÓLICA DO RIO GRANDE DO SUL (PUCRS)

ESCOLA POLITÉCNICA

Algoritmos e Estruturas de Dados II

Turma 320

Guilherme Guimarães Santos

TRABALHO 1 - NÚMEROS DECOMPOSTOS

Relatório apresentado à disciplina de Algoritmos e Estruturas de Dados II, da Escola Politécnica da Pontifícia Universidade Católica do Rio Grande do Sul (PUCRS), como requisito do Trabalho 1, sob orientação do professor João Batista Souza de Oliveira.

Porto Alegre

09 de setembro de 2026

Resumo

Este trabalho trata do problema de encontrar decomposições de números inteiros, ou seja, formas de escrever um número como soma de inteiros positivos e consecutivos. O objetivo era descobrir, em sete faixas de números entre 2 e 2.000.000, qual ou quais números têm a maior quantidade de decomposições, usando uma solução recursiva.

Durante o desenvolvimento, percebemos que a quantidade de decomposições de um número é igual à quantidade de divisores ímpares desse número. Essa relação permitiu construir um algoritmo recursivo eficiente, bem mais rápido do que a primeira versão testada, que apenas testava cada possível ponto de partida da soma. As sete faixas pedidas foram resolvidas em poucos segundos, e os resultados foram conferidos de forma independente através da fatoração dos números encontrados.

1. Introdução

O problema tratado neste trabalho é encontrar decomposições de números inteiros. Uma decomposição de um número n é uma sequência de inteiros positivos e consecutivos cuja soma é igual a n. Por exemplo, o número 15 tem quatro decomposições diferentes:

$$15 = 1 + 2 + 3 + 4 + 5$$

$$15 = 4 + 5 + 6$$

$$15 = 7 + 8$$

$$15 = 15$$

O objetivo deste trabalho é, para sete faixas de números entre 2 e 2.000.000, descobrir qual ou quais números possuem a maior quantidade de decomposições. O enunciado exige que a solução seja implementada de forma recursiva.

Ao estudar o problema, percebemos que a quantidade de decomposições de um número está relacionada à quantidade de divisores ímpares desse número. Essa relação foi essencial para chegar a uma solução eficiente e é explicada na próxima seção.

Este relatório está organizado da seguinte forma: a Seção 2 mostra como o problema foi modelado; a Seção 3 apresenta o algoritmo utilizado, incluindo uma versão ingênua mais simples que foi descartada; a Seção 4 analisa a eficiência das duas versões; a Seção 5 mostra os resultados obtidos para as sete faixas pedidas; e a Seção 6 traz as conclusões.

2. Modelagem

Uma decomposição de n em k termos consecutivos começando em a pode ser escrita como:

$$n = a + (a+1) + (a+2) + \ldots + (a+k-1)$$

Como são k termos, essa soma é igual a:

$$n = k \cdot a + k(k-1)/2$$

Multiplicando os dois lados por 2:

$$2n = k \cdot (2a + k - 1)$$

Ou seja, achar uma decomposição de n é o mesmo que achar uma forma de escrever 2n como o produto de dois números, k e (2a+k-1). A diferença entre esses dois números, $(2a+k-1) - k = 2a-1$, é sempre ímpar, o que significa que, dos dois, um é sempre par e o outro sempre ímpar.

O fator ímpar dessa dupla é, então, um divisor ímpar de 2n. E os divisores ímpares de 2n são exatamente os divisores ímpares de n, já que multiplicar por 2 não cria nem elimina divisores ímpares. Ou seja: cada divisor ímpar de n dá origem a uma decomposição.

Concluindo: **a quantidade de decomposições de n é igual à quantidade de divisores ímpares de n.**

Isso pode ser conferido no próprio exemplo do enunciado: $15 = 3 \times 5$, e seus divisores são 1, 3, 5 e 15, todos ímpares porque 15 é ímpar. São 4 divisores, o mesmo número de decomposições mostrado no enunciado.

Essa relação foi a base da solução: em vez de testar sequências de números consecutivos, o algoritmo conta os divisores ímpares de cada número.

3. Algoritmo

3.1 Versão ingênua

A primeira ideia foi bem direta: para cada n, testar cada possível ponto de partida de 1 até n, e para cada um ir somando os próximos termos até alcançar ou ultrapassar n.

```
função contarIngenuo(n):
    retorna contarComecosPossiveis(n, 1)


função contarComecosPossiveis(n, inicio):
    se inicio > n:
        retorna 0
    se alcancaSoma(n, inicio, inicio, inicio):
        valido = 1
    senão:
```

```
        valido = 0
    retorna valido + contarComecosPossiveis(n, inicio
+ 1)


função alcancaSoma(n, inicio, atual, soma):
    se soma == n:
        retorna verdadeiro
    se soma > n:
        retorna falso
    retorna alcancaSoma(n, inicio, atual + 1, soma +
atual + 1)
```

Essa versão funciona, mas é lenta: para cada n, testa até n pontos de partida diferentes, e cada um deles soma termo a termo. Além de lenta, a recursão tem profundidade proporcional a n (uma chamada de contarComecosPossiveis para cada ponto de partida testado), e na prática isso estoura a pilha de execução para n a partir de aproximadamente 64 mil. Por esses dois motivos essa versão foi descartada; os tempos medidos com ela são usados só como comparação na Seção 4.

3.2 Versão final

A versão usada de fato se apoia diretamente na relação mostrada na Seção 2: em vez de testar pontos de partida, percorremos os tamanhos de sequência k = 1, 2, 3, ..., enquanto a menor soma possível para aquele k (1+2+...+k) ainda couber em n.

```
função contarDecomposicoes(n):
    retorna contar(n, 1)


função contar(n, k):
    menorSoma = k(k+1)/2
    se menorSoma > n:
        retorna 0
    resto = n - k(k-1)/2
    se resto mod k == 0:
        valido = 1
    senão:
```

```
        valido = 0
    retorna valido + contar(n, k + 1)
```

Para cada k, existe no máximo uma sequência de k termos consecutivos que soma n (a que começa em a = resto/k), então basta checar se esse a é um inteiro. Como k só precisa crescer até menorSoma ultrapassar n, a profundidade da recursão é proporcional a √n, bem menor que a versão anterior.

Exemplo de funcionamento, para n = 15:

Tabela 1 – Execução do algoritmo para n = 15

k	menorSoma (≤15?)	resto = n − k(k−1)/2	resto mod k	válido?
1	1 (sim)	15	0	sim
2	3 (sim)	14	0	sim
3	6 (sim)	12	0	sim
4	10 (sim)	9	1	não
5	15 (sim)	5	0	sim
6	21 (não)	—	—	para aqui

Total: 4 decomposições válidas (k=1, 2, 3 e 5), o mesmo valor do enunciado.

4. Eficiência

Na versão ingênua, para cada n são testados até n pontos de partida diferentes, e cada um deles soma seus termos um a um até ultrapassar n. Isso faz o tempo crescer muito mais rápido que o tamanho da entrada, além de estourar a pilha de execução para n a partir de aproximadamente 64 mil, como já mostrado na Seção 3.

Na versão final, o valor de k só precisa crescer até a soma mínima (1+2+...+k) ultrapassar n, ou seja, até k chegar perto de √(2n). Isso limita o número de passos por número a uma quantidade bem menor e previsível, o que se reflete diretamente no tempo de execução.

Para comparar as duas versões na prática, ambas foram executadas sobre faixas pequenas (de 500 até 32.000; acima disso a versão ingênua estoura a pilha, como dito na Seção 3):

Tabela 2 – Tempo de execução das duas versões, por tamanho de faixa testado

Tamanho da faixa (2 até N)	Versão ingênua	Versão final
500	0,001s	0,000s
1.000	0,003s	0,000s
2.000	0,011s	0,000s
4.000	0,048s	0,000s
8.000	0,221s	0,002s
16.000	1,074s	0,006s
32.000	4,936s	0,016s

Para os tamanhos menores (500 e 1.000), o tempo da versão final é tão pequeno que passa a ser dominado por ruído de medição da própria JVM, e não pelo algoritmo em si, por isso ele não cresce de forma perfeitamente suave logo no início do gráfico abaixo.

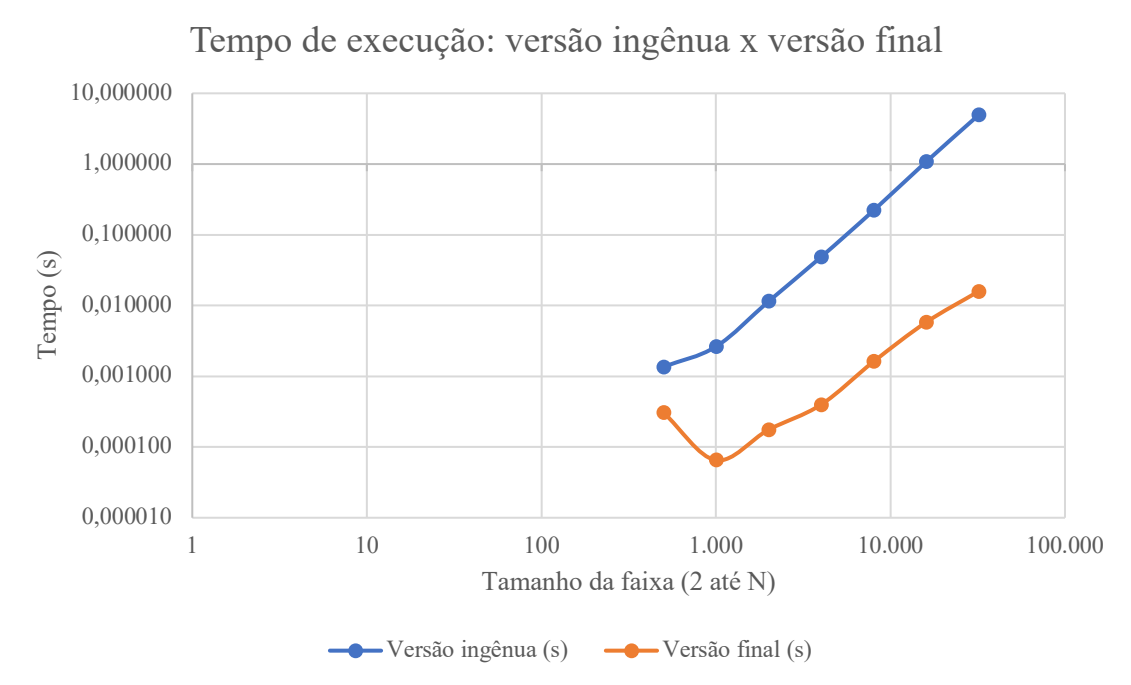


Figura 1 – Tempo (s) por tamanho da faixa testado, escala logarítmica nos dois eixos

Com a versão final, as sete faixas pedidas pelo enunciado, a maior delas indo de 1.500.000 até 2.000.000, foram resolvidas em cerca de 9,1 segundos no total (detalhes

por faixa na Seção 5), o que mostra que a solução escala bem para o tamanho real do problema.

5. Resultados

Aplicando o algoritmo da Seção 3 às sete faixas pedidas pelo enunciado, chegamos aos seguintes resultados:

Tabela 3 – Resultados das sete faixas pedidas pelo enunciado

Faixa	Maior nº de decomposições	Vencedores	Tempo
2 – 100.000	48	19 números (lista abaixo)	0,097s
100.000 – 200.000	64	135135, 176715, 197505	0,181s
200.000 – 400.000	72	8 números (lista abaixo)	0,518s
400.000 – 800.000	96	675675, 765765	1,489s
800.000 – 1.000.000	96	855855, 883575, 945945, 987525	0,920s
1.000.000 – 1.500.000	96	20 números (lista abaixo)	2,716s
1.500.000 – 2.000.000	108	1576575	3,216s

Vencedores completos das faixas com mais de um resultado:

- **2 – 100.000 (48):** 45045, 51975, 58905, 61425, 65835, 69615, 72765, 75075, 77805, 79695, 80325, 85995, 86625, 89775, 90090, 93555, 94185, 96525, 98175
- **200.000 – 400.000 (72):** 225225, 294525, 315315, 329175, 348075, 363825, 389025, 398475
- **1.000.000 – 1.500.000 (96):** 1036035, 1044225, 1119195, 1126125, 1167075, 1195425, 1216215, 1237005, 1276275, 1306305, 1322685, 1351350, 1354815, 1382535, 1396395, 1412775, 1426425, 1461915, 1472625, 1486485

Como validação independente, conferimos a relação mostrada na Seção 2 (decomposições = divisores ímpares) fatorando manualmente um vencedor de cada faixa: por exemplo, $45045 = 3^2 \times 5 \times 7 \times 11 \times 13$, com $(2+1)\times2\times2\times2\times2 = 48$ divisores, o mesmo valor encontrado pelo algoritmo. O mesmo procedimento foi repetido para um vencedor de cada uma das sete faixas, e todos bateram.

As sete faixas juntas, a maior indo até 2.000.000, foram resolvidas em cerca de 9,1 segundos no total.

6. Conclusão

O trabalho foi resolvido a partir da relação entre decomposições e divisores ímpares que apresentamos ao modelar o problema. Essa relação permitiu trocar uma busca por sequências de números consecutivos por uma simples contagem de divisores, o que tornou o algoritmo bem mais rápido do que a primeira versão testada: com apenas 32 mil números, a versão ingênua levou quase 5 segundos, enquanto a versão final resolveu em uma fração de segundo (comparação detalhada na Seção 4).

As sete faixas pedidas pelo enunciado foram resolvidas corretamente. Os resultados foram confirmados de duas formas independentes: pelo próprio algoritmo e pela contagem manual de divisores através da fatoração dos números vencedores.

Como possível melhoria, a versão final ainda calcula a quantidade de decomposições de cada número separadamente, um por um. Uma alternativa seria percorrer os números ímpares como possíveis divisores e, para cada um deles, já ir contando de uma vez em quantos números da faixa ele aparece como divisor, em vez de testar um número por vez. Isso provavelmente seria ainda mais rápido, principalmente para faixas maiores que as pedidas neste trabalho. Não foi necessário aqui, já que os tempos ficaram na casa dos segundos, mas seria um próximo passo natural caso as faixas fossem maiores.

A versão ingênua foi mantida apenas para efeito de comparação de desempenho; ela não faz parte da solução entregue.